

■ Azure Cloud

The Complete Kids' Tutorial

From Zero to Cloud Hero!

30+ Chapters • Hands-On Labs • 3-Tier Capstone Project

Portal • Azure CLI • DevOps • Security • AI Services

■ Cloud Fundamentals	■ Hands-On Labs	■■ 3-Tier Project
■ Security & RBAC	■ Azure DevOps	■ Monitoring

Table of Contents

PART 1 — CLOUD FOUNDATIONS

- Ch 1 — What Is the Cloud?
- Ch 2 — IaaS, PaaS & SaaS Explained
- Ch 3 — Azure vs Other Clouds
- Ch 4 — Creating Your Azure Account

PART 2 — AZURE CORE CONCEPTS

- Ch 5 — Subscriptions & Resource Groups
- Ch 6 — Azure Portal & All Services Overview
- Ch 7 — Azure CLI & Cloud Shell Commands

PART 3 — NETWORKING

- Ch 8 — Virtual Networks (VNETs)
- Ch 9 — Subnets, NSGs & Route Tables

PART 4 — COMPUTE & HOSTING

- Ch 10 — Azure Virtual Machines
- Ch 11 — Azure App Service

PART 5 — STORAGE & DATABASES

- Ch 12 — Azure Blob Storage
- Ch 13 — Azure SQL Server & Database

PART 6 — IDENTITY & SECURITY

- Ch 14 — Azure Entra ID (Azure AD)
- Ch 15 — Azure RBAC
- Ch 16 — Service Principals & Service Connections
- Ch 17 — Azure Security Center

PART 7 — MONITORING & MANAGEMENT

- Ch 18 — Application Insights
- Ch 19 — Azure Monitor
- Ch 20 — Azure Dashboards

PART 8 — DEVOPS & PIPELINES

- Ch 21 — Azure DevOps Overview
- Ch 22 — Azure Repos
- Ch 23 — Azure Boards
- Ch 24 — Azure YAML Pipelines

PART 9 — CAPSTONE PROJECT

- Ch 25 — 3-Tier Architecture — Portal Walkthrough
- Ch 26 — 3-Tier Architecture — Azure CLI Walkthrough

PART 1

Cloud Foundations

Understanding the cloud before we touch Azure

Ch 1: What Is the Cloud?

Ch 2: IaaS, PaaS & SaaS

Ch 3: Azure vs Other Clouds

Ch 4: Creating Your Azure Account

CHAPTER 1

What Is the Cloud?

Imagine the internet as a giant invisible supercomputer you can rent anytime!

■ Think of the cloud like electricity from a power company. You don't build your own power plant — you just plug in and pay for what you use! ■

1.1 The Problem Before the Cloud

Long ago (well, before 2006!), if a company wanted to run a website or store data, they had to buy their own physical servers — big expensive machines kept in a special room called a data center. This was very costly and slow.

- Buying servers cost thousands of dollars upfront.
- If your website got super popular overnight, you couldn't add more power instantly.
- If a server broke, your website went down.
- Companies needed IT teams to maintain hardware 24/7.

1.2 The Solution — Cloud Computing!

Cloud computing means renting computing power, storage, and software over the Internet from big companies like Microsoft, Amazon, or Google. You only pay for what you use — just like a phone plan.

Old Way (On-Premises)	New Way (Cloud)
Buy servers = big upfront cost	Rent servers = pay monthly
Takes weeks to set up	Ready in minutes
Fixed capacity	Scale up/down instantly
You maintain hardware	Cloud provider maintains it
Risk of hardware failure	High availability built in

1.3 Key Cloud Characteristics (NIST Definition)

- **On-demand self-service** — Get resources any time without calling anyone.
- **Broad network access** — Access from laptop, phone, tablet — anywhere!
- **Resource pooling** — Many customers share the same physical hardware safely.
- **Rapid elasticity** — Scale up in seconds, scale down to save money.
- **Measured service** — Pay only for what you consume, like a water bill.

1.4 Types of Cloud Deployment

Type	Who Owns It?	Who Uses It?	Example
Public Cloud	Microsoft/AWS/Google	Anyone on the internet	Azure, AWS
Private Cloud	Your company	Only your company	Company data center
Hybrid Cloud	Both!	Both!	Bank with private + Azure
Multi-Cloud	Multiple providers	One company uses all	Azure + AWS together

■ ■ **ANALOGY:** Public cloud = public library (anyone can use books). Private cloud = your personal bookshelf at home. Hybrid = you keep your diary private but borrow textbooks from the library!

1.5 Why Azure Specifically?

- Microsoft Azure launched in 2010 and is now the #2 cloud provider globally.
- Used by 95% of Fortune 500 companies.
- 60+ regions worldwide (meaning servers close to your users everywhere).
- Deeply integrated with Microsoft tools: Windows, Office 365, GitHub, Visual Studio.
- Strongest enterprise and government compliance certifications.

■ **NOTE** Azure has 200+ services! Don't worry — we'll learn them step by step in this tutorial.

CHAPTER 2

IaaS, PaaS & SaaS Explained

Three flavours of cloud — choose your adventure!

■ ■ **PIZZA ANALOGY:** Making pizza at home (on-premises) vs ordering from a restaurant (SaaS). In between, you can get just the kitchen (IaaS) or the dough + oven (PaaS) and do the rest yourself!

2.1 The Pizza Model of Cloud Services

You Manage	On-Premises	IaaS	PaaS	SaaS
Application	■ You	■ You	■ You	■■ Provider
Data	■ You	■ You	■ You	■■ Provider
Runtime	■ You	■ You	■■ Provider	■■ Provider
Middleware	■ You	■ You	■■ Provider	■■ Provider
OS	■ You	■ You	■■ Provider	■■ Provider
Virtualization	■ You	■■ Provider	■■ Provider	■■ Provider
Hardware	■ You	■■ Provider	■■ Provider	■■ Provider

2.2 IaaS — Infrastructure as a Service

IaaS gives you raw computing infrastructure: virtual machines, storage, and networking. You control the operating system and everything above it. Think of it as renting an empty apartment — the building is taken care of, but you furnish and decorate it yourself.

- **Azure Examples:** Azure Virtual Machines, Azure VNet, Azure Blob Storage
- **Best for:** Dev/test environments, custom software needs, lift-and-shift migrations
- **You manage:** OS updates, middleware, application, data
- **Cloud manages:** Physical servers, network, storage hardware

2.3 PaaS — Platform as a Service

PaaS provides a platform so developers can build, test, and deploy applications without managing underlying infrastructure. It's like renting a fully furnished apartment — just move in and live your life!

- **Azure Examples:** Azure App Service, Azure SQL Database, Azure Functions, Azure Container Apps
- **Best for:** Web apps, APIs, microservices, rapid development

- **You manage:** Application code and data only
- **Cloud manages:** OS, runtime, scaling, patching, networking

2.4 SaaS — Software as a Service

SaaS is ready-to-use software hosted in the cloud. You just open a browser and start using it. Like ordering pizza delivered — you just eat it, no cooking required!

- **Azure/Microsoft Examples:** Microsoft 365, Teams, Dynamics 365, Power BI
- **Best for:** End-users who just need software without any technical setup
- **You manage:** Your data and user settings only
- **Cloud manages:** Everything else!

2.5 Quick Decision Guide

Scenario	Best Choice	Azure Service
I want to run a Windows Server with custom software	IaaS	Azure VM
I want to deploy my web app without managing servers	PaaS	App Service
I need email for my company	SaaS	Microsoft 365
I need a managed database	PaaS	Azure SQL Database
I need to run code only when events happen	PaaS	Azure Functions

CHAPTER 3

Azure vs Other Clouds

Microsoft, Amazon, Google — who wins? Let's compare!

3.1 The Big Three Cloud Providers

Feature	Microsoft Azure	Amazon AWS	Google Cloud (GCP)
Founded	2010	2006	2008
Market Share	~25%	~31%	~11%
Global Regions	60+	32+	38+
Best Known For	Enterprise, Hybrid	Everything, Startups	Big Data, AI/ML
Identity Service	Entra ID (AAD)	IAM	Cloud IAM
VM Service	Azure VMs	EC2	Compute Engine
Kubernetes	AKS	EKS	GKE
Serverless	Azure Functions	AWS Lambda	Cloud Functions
Database	Azure SQL, Cosmos	RDS, DynamoDB	Cloud SQL, Spanner
DevOps	Azure DevOps	CodePipeline	Cloud Build
Free Tier	12 months free + always free	12 months + always free	Always free (no card needed!)

3.2 When to Choose Azure

- Your company already uses Microsoft products (Office 365, Windows Server, Active Directory).
- You need strong compliance for government, healthcare, or finance.
- You want hybrid cloud (connecting on-premises to cloud) — Azure Arc & ExpressRoute shine here.
- You use .NET or Visual Studio for development.
- You need Azure DevOps for end-to-end CI/CD pipelines.

3.3 When to Choose AWS

- You want the largest marketplace of third-party tools.

- You're a startup needing maximum flexibility.
- You need the widest global region coverage.

3.4 When to Choose GCP

- You're doing heavy machine learning and AI research (Google's TPUs!)
- You work heavily with big data using BigQuery.
- You use Kubernetes extensively (Google invented it!).

■ ■ **FUN FACT:** Azure has the highest number of compliance certifications of any cloud provider — 90+ certifications! This is why banks, hospitals, and governments love it.

■ **TIP** Most large companies use multiple clouds (multi-cloud). Learning Azure sets you up perfectly because the concepts transfer to AWS and GCP!

CHAPTER 4

Creating Your Azure Account

Let's get your cloud superpower activated! ■

■ **TIP** Microsoft gives you \$200 free credit for 30 days + 12 months of free services. You need a credit/debit card to verify identity, but you WON'T be charged without your permission!

4.1 What You Need

- A valid email address (use your school or personal email)
- A phone number for verification
- A credit/debit card (for identity only — free tier won't charge you)
- You must be 18+ (or have a parent help you)

4.2 Step-by-Step Account Creation

1	Go to Azure Free Tier page Open your browser and visit azure.microsoft.com/free — click the big green 'Start free' button.
2	Sign in or create a Microsoft account If you have a Hotmail, Outlook, or Microsoft account, sign in. Otherwise click 'Create one!' and make a free account with any email.
3	Verify your identity by phone Enter your phone number. Microsoft will send you a text or call with a verification code. Enter it to continue.
4	Enter credit card details This is ONLY for identity verification. You will NOT be charged. Enter your card details securely on the Microsoft page.
5	Agree to the agreement Read the subscription agreement (it's okay to have a parent help). Check the box and click 'Sign up'.
6	Welcome to Azure! You'll be taken to the Azure Portal at portal.azure.com . Congratulations! You now have a cloud account with \$200 credit! ■

4.3 Exploring the Azure Portal

The Azure Portal (portal.azure.com) is your mission control centre. It's a web-based dashboard where you can create, manage, and monitor all your Azure resources.

- **Top search bar** — Search any service by name (type 'Virtual Machines' to find VMs instantly)
- **Left sidebar** — Quick access to recent resources, favourites, and all services
- **Home page** — Shows your recent resources, quick-create buttons, and Azure news
- **Notifications bell** — See deployment progress and alerts
- **Cloud Shell icon (>_)** — Run Azure CLI commands directly in the browser!
- **Settings gear** — Change portal language, theme (dark mode!), and preferences

4.4 Setting Up Spending Alerts (Very Important!)

■■ **IMPORTANT** Always set a spending alert so you never get surprised by a bill! Follow these steps immediately after creating your account.

1	Go to Cost Management In the portal search bar, type 'Cost Management' and click it.
2	Click 'Budgets' In the left menu, click 'Budgets' then click '+ Add'.
3	Set your budget Name it 'MyBudget'. Set the amount to \$10/month. Click 'Next'.
4	Set alert threshold Set alert at 80% (\$8). Enter your email address. Click 'Create'.
5	You're protected! Azure will email you if your spending approaches \$8. Sleep easy! ■

PART 2

Azure Core Concepts

The building blocks every Azure engineer must know

Ch 5: Subscriptions & Resource Groups

Ch 6: All Services Overview

Ch 7: Azure CLI & Cloud Shell

CHAPTER 5

Subscriptions & Resource Groups

Organising your cloud — like folders for your homework!

■ ■ ■ **ANALOGY:** Azure Tenant = Your school. Subscription = Your classroom. Resource Group = Your school bag. Resources = Your books, pencils, notebooks.

5.1 Azure Management Hierarchy

- **Management Group** — Top level. Groups multiple subscriptions (used by large enterprises).
- **Subscription** — Your billing account. All resources belong to one subscription.
- **Resource Group** — Logical container for related resources (like a folder).
- **Resource** — An actual Azure service (VM, storage account, database, etc.).

5.2 What is an Azure Subscription?

A subscription is the billing boundary in Azure. Every resource you create belongs to a subscription. You can have multiple subscriptions to separate billing, environments (dev vs prod), or departments.

- **Pay-As-You-Go** — Pay for what you use monthly. Great for beginners.
- **Free Trial** — \$200 credit for 30 days. What you just created!
- **Enterprise Agreement** — Large companies negotiate bulk pricing with Microsoft.
- **Visual Studio** — Developers get free monthly credits (\$50-\$150/month!).

5.3 What is a Resource Group?

A Resource Group is a logical container that holds related Azure resources for an application. Think of it as a project folder — when the project is done, you delete the whole folder at once!

- Every resource **MUST** belong to exactly one resource group.
- Resources in different regions can be in the same resource group.
- Deleting a resource group deletes **ALL** resources inside it (very powerful!).
- Use resource groups to organise by: environment (dev/test/prod), project, team, or cost centre.

5.4 Creating a Resource Group — Portal

- 1 Open Resource Groups**
In the portal, search for 'Resource groups' and click it.
- 2 Click '+ Create'**
Click the '+ Create' button at the top left.

3	Fill in details Subscription: select your free trial. Resource group name: type 'rg-myapp-dev'. Region: select '(Asia Pacific) Southeast Asia' or the closest region to you.
4	Review + Create Click 'Review + create', then click 'Create'. Wait a few seconds...
5	Done! ■ Your resource group appears in the list. Click it to explore!

5.5 Creating a Resource Group — Azure CLI

```
AZURE CLI — Resource Group Commands
# Login to Azure first
az login

# Create a resource group
az group create \
  --name rg-myapp-dev \
  --location southeastasia

# List all resource groups
az group list --output table

# Show details of a specific group
az group show --name rg-myapp-dev

# Delete a resource group (careful! deletes everything inside!)
az group delete --name rg-myapp-dev --yes --no-wait
```

5.6 Resource Group Best Practices

- **Naming convention:** rg-{appname}-{environment} → rg-webshop-prod, rg-webshop-dev
- **One group per environment:** Keep dev, test, staging, prod separate.
- **One group per project:** All resources for 'Project Phoenix' in rg-phoenix-prod.
- **Add tags:** Key-value pairs like Environment=Dev, Owner=TeamA, CostCenter=1234.
- **Lock important groups:** Add a delete lock to production groups to prevent accidents!

5.7 Adding Tags to a Resource Group — CLI

```
AZURE CLI — Tags
# Add tags to resource group
az group update \
  --name rg-myapp-dev \
  --tags Environment=Dev Owner=Student CostCenter=Education

# View tags
az group show --name rg-myapp-dev --query tags
```


CHAPTER 6

Azure Portal & All Services Overview

Your mission control centre — 200+ services at your fingertips!

6.1 Azure Service Categories

Category	Key Services	What They Do
Compute	VMs, App Service, AKS, Functions	Run your applications and code
Networking	VNet, Load Balancer, DNS, VPN	Connect resources and users
Storage	Blob, Files, Queues, Tables	Store any type of data
Databases	SQL, Cosmos DB, Redis, PostgreSQL	Managed database services
Identity	Entra ID, RBAC, MFA	Who can access what
Security	Defender, Sentinel, Key Vault	Protect your resources
DevOps	Azure DevOps, Pipelines, Repos	Build and deploy software
Monitoring	Monitor, App Insights, Log Analytics	Watch and alert on everything
AI + ML	OpenAI, Cognitive Services, ML Studio	Add intelligence to apps
Integration	Logic Apps, Service Bus, API Mgmt	Connect apps and services
IoT	IoT Hub, IoT Central, Digital Twins	Connect physical devices
Analytics	Synapse, Data Factory, Power BI	Analyse big data

6.2 Most Important Services for Beginners

- **Resource Groups** — Organise everything (Chapter 5).
- **Virtual Machines** — Rent a computer in the cloud (Chapter 10).
- **App Service** — Host websites without managing servers (Chapter 11).
- **Blob Storage** — Store files, images, videos cheaply (Chapter 12).
- **Azure SQL** — Managed relational database (Chapter 13).
- **Entra ID** — Manage users and permissions (Chapter 14).
- **Key Vault** — Store secrets and passwords safely.
- **Application Insights** — Monitor your app's performance (Chapter 18).

6.3 Azure Marketplace

The Azure Marketplace is like an app store for the cloud. You can find pre-built solutions from Microsoft and partners: WordPress, Minecraft Server, Machine Learning models, security tools, and much more — often deployable in one click!

■ **TIP** Try this: In the portal, search 'Marketplace' and type 'WordPress'. You can deploy a full WordPress website in about 5 minutes!

Azure CLI & Cloud Shell Commands

7.1 What is Azure CLI?

7.2 Installing Azure CLI

7.3 Essential Azure CLI Commands

```
# ■■ Storage ████████████████████████████████████████████
```

```
az storage account create --name mystorageXXXX \  
--resource-group rg-demo --location eastus \  
--sku Standard_LRS  
  
az storage account list --output table  
  
# ■■ App Service ████████████████████████████████████████████
```

```
az appservice plan create --name myPlan \  
--resource-group rg-demo --sku FREE  
  
az webapp create --resource-group rg-demo \  
--plan myPlan --name myapp12345 --runtime 'NODE:18-lts'  
  
# ■■ Help & Docs ████████████████████████████████████████████
```

```
az --help # Top-level help  
  
az vm --help # VM subcommand help  
  
az vm create --help # Specific command help  
  
az find 'create storage' # AI-powered search!
```

7.4 Using Azure Cloud Shell

Azure Cloud Shell is a browser-based terminal built into the Azure Portal. No installation needed — just click the '>' icon at the top of the portal!

- Choose Bash or PowerShell (both work for Azure CLI).
- Files are automatically saved between sessions.
- Comes pre-installed with Azure CLI, kubectl, Terraform, and more.
- FREE to use — you only pay for the storage it uses (< \$1/month).

- 1 Open Cloud Shell**
In the Azure Portal, click the '>_' icon in the top navigation bar.
- 2 Choose Bash**
Select 'Bash' when prompted (or PowerShell if you prefer).
- 3 Create storage**
Click 'Create storage' — Azure creates a small file share for your session files.
- 4 Start typing!**
Type 'az account show' and press Enter. You're running CLI in the cloud!

7.5 Azure PowerShell Equivalents

```
AZURE POWERSHELL
# Install Az PowerShell module
Install-Module -Name Az -AllowClobber -Scope CurrentUser
# Login
Connect-AzAccount
```

```
# Get resource groups
Get-AzResourceGroup | Format-Table

# Create resource group
New-AzResourceGroup -Name rg-demo -Location 'East US'

# Remove resource group
Remove-AzResourceGroup -Name rg-demo -Force
```


PART 3

Networking

Building your private network in the cloud

Ch 8: Virtual Networks (VNETs)

Ch 9: Subnets, NSGs & Route Tables

CHAPTER 8

Virtual Networks (VNets)

Your own private neighbourhood in the cloud! ■■

■■■ **ANALOGY:** A VNet is like a gated neighbourhood. The houses (VMs, App Services) inside can talk to each other. Outsiders can only enter through the gate (public IP / load balancer). Different neighbourhoods need bridges (VNet peering) to talk.

8.1 What is a VNet?

An Azure Virtual Network (VNet) is your own private, isolated network in the Azure cloud. You define the IP address space, create subnets, control traffic, and connect resources securely. Think of it as drawing your own network on paper — but in the cloud!

- VNets are region-specific (one VNet = one Azure region).
- You define an IP address range using CIDR notation, e.g., 10.0.0.0/16.
- Resources in the same VNet can communicate by default.
- VNets can be connected to each other (VNet Peering) or to on-premises (VPN/ExpressRoute).

8.2 CIDR Notation Explained

CIDR (Classless Inter-Domain Routing) notation describes IP ranges. The number after the slash tells you how many IP addresses are in the range.

CIDR	Hosts Available	Use Case
10.0.0.0/8	16,777,216 IPs	Very large enterprise
10.0.0.0/16	65,536 IPs	Large VNet (recommended)
10.0.0.0/24	256 IPs	Single subnet
10.0.0.0/28	16 IPs	Small test subnet

8.3 Creating a VNet — Portal

- 1 Search for Virtual Networks**
In the portal search bar, type 'Virtual Networks' and click it.
- 2 Click '+ Create'**
Click '+ Create' to start the VNet creation wizard.

3	Basics tab Select your subscription and resource group (rg-myapp-dev). Name: 'vnet-myapp-dev'. Region: same as your resource group.
4	IP Addresses tab Keep the default 10.0.0.0/16 address space. Delete the default subnet and we'll add proper ones in Chapter 9.
5	Security tab For now, leave BastionHost, DDoS Protection, and Firewall as disabled (they cost extra).
6	Review + Create Click 'Review + create', then 'Create'. Takes about 30 seconds!

8.4 Creating a VNet — Azure CLI

```
AZURE CLI — Virtual Network
# Create a Virtual Network
az network vnet create \
--resource-group rg-myapp-dev \
--name vnet-myapp-dev \
--address-prefix 10.0.0.0/16 \
--location southeastasia

# List VNets
az network vnet list --output table

# Show VNet details
az network vnet show \
--resource-group rg-myapp-dev \
--name vnet-myapp-dev

# Delete a VNet
az network vnet delete \
--resource-group rg-myapp-dev \
--name vnet-myapp-dev
```

8.5 VNet Peering

VNet Peering connects two VNets so resources can communicate as if they were on the same network. Both VNets must have non-overlapping IP ranges.

```
AZURE CLI — VNet Peering
# Create peering from vnet1 to vnet2
az network vnet peering create \
--resource-group rg-myapp-dev \
--name vnet1-to-vnet2 \
--vnet-name vnet-myapp-dev \
```

```
--remote-vnet vnet-myapp-prod \  
--allow-vnet-access
```

CHAPTER 9

Subnets, NSGs & Route Tables

Dividing your neighbourhood into streets with traffic rules! ■

9.1 Subnets

A subnet is a smaller division of your VNet's IP space. You use subnets to organise resources by tier (web, app, database) and apply different security rules to each.

Subnet Name	CIDR	Purpose
subnet-web	10.0.1.0/24	Public-facing web servers / App Service
subnet-app	10.0.2.0/24	Application layer / backend APIs
subnet-db	10.0.3.0/24	Databases (most restricted)
subnet-mgmt	10.0.4.0/28	Management / Bastion hosts

AZURE CLI — Subnets

```
# Add subnets to your VNet
```

```
az network vnet subnet create \
```

```
--resource-group rg-myapp-dev \
```

```
--vnet-name vnet-myapp-dev \
```

```
--name subnet-web \
```

```
--address-prefix 10.0.1.0/24
```

```
az network vnet subnet create \
```

```
--resource-group rg-myapp-dev \
```

```
--vnet-name vnet-myapp-dev \
```

```
--name subnet-db \
```

```
--address-prefix 10.0.3.0/24
```

```
# List subnets
```

```
az network vnet subnet list \
```

```
--resource-group rg-myapp-dev \
```

```
--vnet-name vnet-myapp-dev \
```

```
--output table
```

9.2 Network Security Groups (NSGs)

An NSG is like a firewall for your subnet or network interface. It contains rules that allow or deny inbound and outbound traffic. Each rule has: priority, source, destination, port, protocol, and action (Allow/Deny).

AZURE CLI — NSG Rules

```
# Create a Network Security Group
```

```

az network nsg create \
--resource-group rg-myapp-dev \
--name nsg-web

# Add rule: Allow HTTP from internet
az network nsg rule create \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--name Allow-HTTP \
--priority 100 \
--direction Inbound \
--source-address-prefixes Internet \
--destination-port-ranges 80 \
--protocol Tcp \
--access Allow

# Add rule: Allow HTTPS
az network nsg rule create \
--resource-group rg-myapp-dev \
--nsg-name nsg-web \
--name Allow-HTTPS \
--priority 110 \
--direction Inbound \
--source-address-prefixes Internet \
--destination-port-ranges 443 \
--protocol Tcp \
--access Allow

# Attach NSG to subnet
az network vnet subnet update \
--resource-group rg-myapp-dev \
--vnet-name vnet-myapp-dev \
--name subnet-web \
--network-security-group nsg-web

```

■ **IMPORTANT** NSG rules are evaluated in order of priority. Lower number = higher priority. If no rule matches, default rules apply (deny all inbound, allow all outbound).

PART 4

Compute & Hosting

Running your applications and APIs

Ch 10: Azure Virtual Machines

Ch 11: Azure App Service

CHAPTER 10

Azure Virtual Machines

Rent a computer in the cloud — any size, any OS! ■

■ ■ A VM is like a Minecraft server in the cloud — you choose the size, install what you want, and control everything! Shut it down at night and save money.

10.1 VM Size Series

Series	Purpose	Example Size	vCPU / RAM
B-series	Burstable, dev/test, free tier	Standard_B1s	1 vCPU / 1 GB
D-series	General purpose, web servers	Standard_D2s_v3	2 vCPU / 8 GB
E-series	Memory optimised, databases	Standard_E4s_v3	4 vCPU / 32 GB
F-series	Compute optimised, game servers	Standard_F4s_v2	4 vCPU / 8 GB
N-series	GPU, machine learning, gaming	Standard_NC6	6 vCPU + 1 GPU

10.2 Creating a VM — Portal (Step by Step)

1 Search for Virtual Machines

In the portal, search 'Virtual machines' and click it. Then click '+ Create' → 'Azure virtual machine'.

2 Basics tab — Project Details

Subscription: your free trial. Resource group: rg-myapp-dev.

3 Basics tab — Instance Details

VM name: vm-webserver-01. Region: Southeast Asia. Availability: No infrastructure redundancy. Image: Ubuntu Server 22.04 LTS. Size: Standard_B1s (free tier eligible!).

4 Basics tab — Administrator Account

Authentication type: SSH public key. Username: azureuser. SSH key: Generate new key pair (download and save it!).

5	Inbound Ports Allow SSH (22) and HTTP (80). We'll tighten this with NSGs later.
6	Disks tab OS disk type: Standard SSD (cheaper). Leave other defaults.
7	Networking tab Virtual network: vnet-myapp-dev. Subnet: subnet-web. Public IP: Create new. NIC NSG: Basic.
8	Management tab Enable auto-shutdown at 11 PM to save money! Set your timezone.
9	Review + Create Review the estimated cost. Click 'Create'. Download your SSH key when prompted!
10	Wait for deployment Takes 2-3 minutes. You'll see a notification when complete. Click 'Go to resource'.

10.3 Connect to Your VM via SSH

```
CONNECT & CONFIGURE YOUR VM
# Make your SSH key private (Linux/Mac only)
chmod 400 ~/Downloads/mykey.pem
# Connect to VM (replace PUBLIC_IP with your VM's IP)
ssh -i ~/Downloads/mykey.pem azureuser@PUBLIC_IP
# Once connected, update packages
sudo apt update && sudo apt upgrade -y
# Install nginx web server
sudo apt install nginx -y
# Start nginx
sudo systemctl start nginx
sudo systemctl enable nginx
# Test - open browser: http://YOUR_PUBLIC_IP
# You should see the nginx welcome page!
```

10.4 VM Operations — Azure CLI

```
AZURE CLI - VM Lifecycle
# Create a VM
az vm create \
  --resource-group rg-myapp-dev \
  --name vm-webserver-01 \
  --image Ubuntu2204 \
```



```
--size Standard_B1s \  
--admin-username azureuser \  
--generate-ssh-keys \  
--public-ip-sku Standard  
  
# Open port 80 for web traffic  
az vm open-port --port 80 \  
--resource-group rg-myapp-dev \  
--name vm-webserver-01  
  
# Get public IP of VM  
az vm list-ip-addresses \  
--resource-group rg-myapp-dev \  
--name vm-webserver-01 \  
--output table  
  
# Stop VM (saves compute cost, storage still billed)  
az vm stop --resource-group rg-myapp-dev --name vm-webserver-01  
  
# Deallocate VM (stops ALL billing except disk)  
az vm deallocate --resource-group rg-myapp-dev --name vm-webserver-01  
  
# Start VM  
az vm start --resource-group rg-myapp-dev --name vm-webserver-01  
  
# Delete VM  
az vm delete --resource-group rg-myapp-dev --name vm-webserver-01 --yes
```

■ **TIP** Always DEALLOCATE (not just stop) your VM when not using it to stop compute charges. The disk still costs a few cents, but compute stops.

CHAPTER 11

Azure App Service

Deploy websites and APIs in minutes — no server management! ■

11.1 What is App Service?

Azure App Service is a fully managed platform for building, deploying, and scaling web apps. You just push your code and Azure handles the server, OS updates, patching, load balancing, and auto-scaling. It's PaaS at its best!

- Supports .NET, Node.js, Python, Java, PHP, Ruby out of the box.
- Deploy from GitHub, Azure DevOps, Docker, or FTP.
- Built-in CI/CD integration.
- Auto-scale based on HTTP traffic.
- FREE tier available (F1) with 1GB storage, 60 CPU minutes/day.

11.2 App Service Plan

An App Service Plan defines the compute resources for your web app. Multiple apps can share one plan. The plan determines CPU, RAM, scaling, and price.

Tier	Name	vCores	RAM	Custom Domain	Auto-scale
Free	F1	Shared	1 GB	■	■
Shared	D1	Shared	1 GB	■	■
Basic	B1	1	1.75 GB	■	■
Standard	S1	1	1.75 GB	■	■ (up to 10)
Premium	P1v3	2	8 GB	■	■ (up to 30)

11.3 Creating an App Service — Portal

1

Search App Service

Search 'App Services' in the portal and click '+ Create'.

2

Project Details

Subscription: free trial. Resource group: rg-myapp-dev.

3	Instance Details Name: mywebapp-{yourname}-dev (must be globally unique!). Publish: Code. Runtime stack: Node 18 LTS. OS: Linux. Region: Southeast Asia.
4	App Service Plan Click 'Create new'. Name: asp-myapp-dev. SKU: Free F1 (click 'Explore pricing plans' to see all options).
5	Review + Create Review and click 'Create'. Deployment takes about 1-2 minutes.
6	Test your app Go to the App Service URL shown: https://mywebapp-yourname-dev.azurewebsites.net — you'll see a default page!

11.4 Deploy Code to App Service — CLI

```

AZURE CLI — App Service
# Create App Service Plan
az appservice plan create \
--name asp-myapp-dev \
--resource-group rg-myapp-dev \
--sku FREE \
--is-linux

# Create Web App
az webapp create \
--resource-group rg-myapp-dev \
--plan asp-myapp-dev \
--name mywebapp-demo-dev \
--runtime 'NODE:18-lts'

# Deploy code from a local folder
az webapp up \
--resource-group rg-myapp-dev \
--name mywebapp-demo-dev \
--html

# Set environment variable (like a secret config)
az webapp config appsettings set \
--resource-group rg-myapp-dev \
--name mywebapp-demo-dev \
--settings DATABASE_URL='myconnectionstring' NODE_ENV='production'

# Browse the app
az webapp browse \
--resource-group rg-myapp-dev \
--name mywebapp-demo-dev

# View live logs (streaming)

```

```
az webapp log tail \  
--resource-group rg-myapp-dev \  
--name mywebapp-demo-dev  
  
# Delete the web app  
az webapp delete \  
--resource-group rg-myapp-dev \  
--name mywebapp-demo-dev
```

11.5 Deployment Slots

Deployment slots let you have multiple versions of your app (staging, production) on the same App Service. You can swap slots instantly with zero downtime — test in staging, then swap to production!

```
DEPLOYMENT SLOTS  
# Create a staging slot (requires Standard+ tier)  
az webapp deployment slot create \  
--resource-group rg-myapp-dev \  
--name mywebapp-demo-dev \  
--slot staging  
  
# Swap staging to production  
az webapp deployment slot swap \  
--resource-group rg-myapp-dev \  
--name mywebapp-demo-dev \  
--slot staging
```


PART 5

Storage & Databases

Keeping your data safe and accessible

Ch 12: Azure Blob Storage

Ch 13: Azure SQL Database

CHAPTER 12

Azure Blob Storage

Store any file in the cloud — cheap, durable, and globally accessible! ■

■ ■ ■ **ANALOGY:** Azure Blob Storage is like a giant Google Drive for your app. Images, videos, backups, log files — anything goes in, accessible from anywhere in the world.

12.1 Storage Account Basics

A Storage Account is the parent container for all Azure Storage services. Inside one account you can have: Blob Storage (files), File Shares (like a network drive), Queues (messages), and Tables (NoSQL data).

Storage Type	Like...	Use Case
Blob Storage	File system / S3	Images, videos, backups, static websites
File Share	Network drive (SMB)	Lift-and-shift file servers
Queue Storage	Message queue	Async processing, task queues
Table Storage	NoSQL key-value	User data, logging

12.2 Blob Container Structure

- **Storage Account** — Top level (globally unique name, max 24 chars).
- **Container** — Like a folder inside the storage account.
- **Blob** — The actual file (can be any size up to 190 TB!).

12.3 Access Tiers

Tier	Access Frequency	Storage Cost	Access Cost
Hot	Frequent	Highest	Lowest
Cool	Infrequent (>30 days)	Lower	Moderate
Cold	Rare (>90 days)	Even lower	Higher
Archive	Very rare (>180 days)	Cheapest	Highest (rehydrate needed)

12.4 Creating Blob Storage — Portal

1	Search Storage Accounts Search 'Storage accounts' and click '+ Create'.
2	Basics Resource group: rg-myapp-dev. Storage account name: mystorageXXXX (use your name, must be unique globally). Region: Southeast Asia. Performance: Standard. Redundancy: LRS (Locally Redundant — cheapest).
3	Advanced tab Enable 'Allow enabling anonymous access on individual containers' if you want public image hosting.
4	Review + Create Click 'Create'. Takes 20-30 seconds.
5	Create a Container Go to your storage account → 'Containers' in left menu → '+ Container'. Name: 'images'. Access level: Blob (public read for blobs).
6	Upload a file Click your container, then 'Upload'. Choose any image file from your computer!
7	Get the URL Click the uploaded blob. Copy the URL. Open it in a browser — your file is public on the internet! ■

12.5 Blob Storage — Azure CLI

```

AZURE CLI — Blob Storage
# Create storage account
az storage account create \
  --name mystorageXXXX \
  --resource-group rg-myapp-dev \
  --location southeastasia \
  --sku Standard_LRS \
  --kind StorageV2 \
  --access-tier Hot
# Create a container
az storage container create \
  --account-name mystorageXXXX \
  --name images \
  --public-access blob
# Upload a file
az storage blob upload \
  --account-name mystorageXXXX \

```



```
--container-name images \
--name myphoto.jpg \
--file ~/myphoto.jpg
```

```
# List blobs in container
```

```
az storage blob list \
--account-name mystorageXXXX \
--container-name images \
--output table
```

```
# Download a blob
```

```
az storage blob download \
--account-name mystorageXXXX \
--container-name images \
--name myphoto.jpg \
--file ~/downloaded.jpg
```

```
# Delete a blob
```

```
az storage blob delete \
--account-name mystorageXXXX \
--container-name images \
--name myphoto.jpg
```

```
# Delete storage account
```

```
az storage account delete \
--name mystorageXXXX \
--resource-group rg-myapp-dev \
--yes
```

CHAPTER 13

Azure SQL Server & Database

Managed relational databases — all the power, none of the admin! ■■

13.1 Azure SQL vs SQL Server on VM

Feature	Azure SQL Database (PaaS)	SQL Server on VM (IaaS)
Management	Fully managed	You manage OS + SQL
Patching	Automatic	Manual
Backups	Automatic (up to 35 days)	You configure
Scale	Click to scale	Resize VM
HA/DR	Built-in	Configure AlwaysOn
Cost	Pay per DTU/vCore	Pay for VM + license
Best for	New apps, dev/test	Lift-and-shift, full control

13.2 Creating Azure SQL — Portal

1	Search SQL databases Search 'SQL databases' in the portal and click '+ Create'.
2	Basics Resource group: rg-myapp-dev. Database name: db-myapp-dev.
3	Create a server Click 'Create new' under Server. Server name: sql-myapp-dev-XXX (globally unique). Location: Southeast Asia. Authentication: SQL authentication. Set admin username: sqladmin and a strong password.
4	Compute + storage Click 'Configure database'. Choose 'Basic' (5 DTUs, ~\$5/month for learning). Click Apply.
5	Networking tab Connectivity method: Public endpoint. Add current client IP (for testing from your PC). Allow Azure services: Yes.

6 Review + Create

Click Create. Takes 2-5 minutes.

7 Connect with Azure Data Studio

Download Azure Data Studio (free Microsoft tool). Create new connection using your server name (.database.windows.net), admin login, and password.

13.3 Azure SQL — CLI

AZURE CLI — SQL Database

```
# Create SQL Server
```

```
az sql server create \
```

```
--name sql-myapp-dev-XXX \
```

```
--resource-group rg-myapp-dev \
```

```
--location southeastasia \
```

```
--admin-user sqladmin \
```

```
--admin-password 'MyP@ssw0rd123!'
```

```
# Allow your IP to connect
```

```
az sql server firewall-rule create \
```

```
--resource-group rg-myapp-dev \
```

```
--server sql-myapp-dev-XXX \
```

```
--name AllowMyIP \
```

```
--start-ip-address 0.0.0.0 \
```

```
--end-ip-address 255.255.255.255
```

```
# Create a database
```

```
az sql db create \
```

```
--resource-group rg-myapp-dev \
```

```
--server sql-myapp-dev-XXX \
```

```
--name db-myapp-dev \
```

```
--edition Basic \
```

```
--capacity 5
```

```
# List databases on a server
```

```
az sql db list \
```

```
--resource-group rg-myapp-dev \
```

```
--server sql-myapp-dev-XXX \
```

```
--output table
```

```
# Delete database
```

```
az sql db delete \
```

```
--resource-group rg-myapp-dev \
```

```
--server sql-myapp-dev-XXX \
```

```
--name db-myapp-dev --yes
```

```
# Delete SQL Server
```

```
az sql server delete \
```

```
--resource-group rg-myapp-dev \
```

```
--name sql-myapp-dev-XXX --yes
```


PART 6

Identity & Security

Controlling who can do what in your cloud

Ch 14: Azure Entra ID

Ch 15: Azure RBAC

Ch 16: Service Principals

Ch 17: Azure Security

CHAPTER 14

Azure Entra ID (Azure Active Directory)

The bouncer at the door of your cloud! ■

■ ■ **ANALOGY:** Entra ID is like the school office that gives everyone their student ID card. Every person or app that wants to do anything in Azure must show their ID card first!

14.1 What is Azure Entra ID?

Azure Entra ID (formerly Azure Active Directory / Azure AD) is Microsoft's cloud-based identity and access management service. It authenticates WHO you are and authorises WHAT you can do. Every Azure subscription has a tenant (an Entra ID directory).

- **Users** — Human accounts (employees, admins, guests).
- **Groups** — Collections of users to apply permissions at scale.
- **Applications** — Apps registered to access Azure resources.
- **Service Principals** — App identities (see Chapter 16).
- **Managed Identities** — Azure service identities (no password needed!).

14.2 Creating a User — Portal

1	Open Entra ID Search 'Microsoft Entra ID' in the portal.
2	Click Users In the left menu, click 'Users' then '+ New user' → 'Create new user'.
3	Fill details User principal name: jsmith@yourtenantname.onmicrosoft.com. Display name: John Smith. Password: Auto-generate (or set manually).
4	Assignments Click 'Next: Properties'. Add job title, department. Click 'Next: Assignments'.
5	Add to groups Click '+ Add group'. Add the user to a group like 'Developers'.
6	Review + Create Click 'Review + create' then 'Create'. Done! John can now log into Azure.

14.3 Creating a Group — CLI

```
AZURE CLI -- Entra ID
# Create a security group
az ad group create \
--display-name 'Developers' \
--mail-nickname 'developers'
# List groups
az ad group list --output table
# Add user to group
az ad group member add \
--group 'Developers' \
--member-id <user-object-id>
# Create a new user
az ad user create \
--display-name 'Jane Doe' \
--user-principal-name jdoe@yourtenant.onmicrosoft.com \
--password 'TempP@ss123!'
```

14.4 Multi-Factor Authentication (MFA)

MFA requires users to verify their identity with two factors: something they know (password) + something they have (phone app code). Enable MFA for all admin accounts!

1 Go to Entra ID

Search 'Microsoft Entra ID' in the portal.

2 Security > MFA

Click 'Security' in the left menu, then 'Multifactor authentication'.

3 Per-user MFA

Click 'Users' and enable MFA for specific users, or use Conditional Access for more advanced policies.

CHAPTER 15

Azure RBAC

Role-Based Access Control — the right key for the right door! ■■

■■ **ANALOGY:** RBAC is like a castle with different key cards. The King (Owner) can do everything. The Knight (Contributor) can fight but not make new knights. The Messenger (Reader) can only look around, not touch anything.

15.1 RBAC Concepts

- **Security Principal** — Who gets the role: User, Group, Service Principal, or Managed Identity.
- **Role Definition** — What actions are allowed: Owner, Contributor, Reader, or custom.
- **Scope** — Where it applies: Management Group, Subscription, Resource Group, or Resource.
- **Role Assignment** — Combining the above three: 'Give Jane Contributor access to rg-myapp-dev'.

15.2 Built-in Roles

Role	Can Create?	Can Read?	Can Delete?	Manage Access?
Owner	■	■	■	■
Contributor	■	■	■	■
Reader	■	■	■	■
User Access Admin	■	■	■	■

15.3 Assigning a Role — Portal

1	Go to Resource Group Navigate to your resource group (rg-myapp-dev).
2	Click Access Control (IAM) In the left menu, click 'Access control (IAM)'.
3	Add role assignment Click '+ Add' → 'Add role assignment'.
4	Select Role Search for 'Reader'. Select it. Click 'Next'.

5 Select Members

Click '+ Select members'. Search for the user or group. Click 'Select'.

6 Review + Assign

Review and click 'Review + assign'. Done! The user can now read resources in this group.

15.4 RBAC — Azure CLI

```
AZURE CLI — RBAC
# List current role assignments in a resource group
az role assignment list \
--resource-group rg-myapp-dev \
--output table

# Assign Reader role to a user
az role assignment create \
--assignee jdoe@yourtenant.onmicrosoft.com \
--role 'Reader' \
--resource-group rg-myapp-dev

# Assign Contributor role to a group
az role assignment create \
--assignee <group-object-id> \
--role 'Contributor' \
--resource-group rg-myapp-dev

# Remove a role assignment
az role assignment delete \
--assignee jdoe@yourtenant.onmicrosoft.com \
--role 'Reader' \
--resource-group rg-myapp-dev

# List all available built-in roles
az role definition list --output table | head -50
```

■ **TIP** Principle of Least Privilege: Always assign the minimum permissions needed. Start with Reader, add more only when required. Never give everyone Owner access!

CHAPTER 16

Service Principals & Service Connections

Robot identity cards for your automation! ■

16.1 What is a Service Principal?

A Service Principal is an identity for an application or service to access Azure resources. Instead of using your personal login in a pipeline or script, you create a Service Principal — like a robot employee with its own ID card and specific permissions.

- Used by: CI/CD pipelines (GitHub Actions, Azure DevOps), scripts, apps.
- Has a client ID (username) and client secret or certificate (password).
- You assign RBAC roles to it, just like a user.
- **Managed Identity** is even better — Azure manages the secret for you, no password to store!

16.2 Creating a Service Principal — CLI

```
AZURE CLI — Service Principal
# Create a service principal with Contributor role on a subscription
az ad sp create-for-rbac \
--name sp-myapp-dev \
--role Contributor \
--scopes /subscriptions/<your-subscription-id>
# Output looks like:
# {
#   'appId': '...', <- This is your CLIENT_ID
#   'displayName': '...',
#   'password': '...', <- This is your CLIENT_SECRET (save it now!)
#   'tenant': '...' <- This is your TENANT_ID
# }
# Create SP scoped to a resource group (better practice!)
az ad sp create-for-rbac \
--name sp-myapp-rg \
--role Contributor \
--scopes /subscriptions/<sub-id>/resourceGroups/rg-myapp-dev
# List service principals
az ad sp list --display-name sp-myapp-dev --output table
# Delete service principal
az ad sp delete --id <appId>
```

■ **IMPORTANT** The client secret is shown ONLY ONCE when created. Copy and save it immediately in Azure Key Vault or a password manager. Never put it in your code or Git!

16.3 Service Connection in Azure DevOps

A Service Connection in Azure DevOps lets your pipeline authenticate to Azure without embedding credentials in code. It uses the Service Principal under the hood.

1	Open Azure DevOps Go to dev.azure.com and open your project.
2	Project Settings Click 'Project settings' (bottom left gear icon).
3	Service connections Under Pipelines section, click 'Service connections'.
4	New service connection Click '+ New service connection' → 'Azure Resource Manager'.
5	Authentication method Select 'Service principal (automatic)' for easiest setup, or 'Service principal (manual)' to use one you created.
6	Configure Subscription: select yours. Resource group: leave blank (subscription-level) or select specific group. Name: 'sc-azure-myapp'. Check 'Grant access permission to all pipelines'. Click Save.
7	Use in pipeline In your YAML pipeline, reference: azureSubscription: 'sc-azure-myapp'

CHAPTER 17

Azure Security

Locking the doors, windows, and secret passages! ■■

17.1 Azure Security Center / Microsoft Defender for Cloud

Microsoft Defender for Cloud is your unified security management dashboard. It continuously assesses your resources and gives you a 'Secure Score' — like a security report card!

- **Secure Score** — A percentage showing how well your environment is secured. Higher = safer!
- **Recommendations** — Specific actions to improve security ('Enable MFA for all admin users').
- **Compliance** — Check compliance against standards like ISO 27001, PCI DSS, HIPAA.
- **Threat protection** — Detect attacks in real-time (requires paid tier).

17.2 Azure Key Vault

Azure Key Vault is a secure, managed store for secrets (passwords, connection strings, API keys), encryption keys, and certificates. Never hard-code secrets in your code — use Key Vault!

```
AZURE KEY VAULT
# Create a Key Vault
az keyvault create \
--name kv-myapp-dev-XXX \
--resource-group rg-myapp-dev \
--location southeastasia

# Store a secret
az keyvault secret set \
--vault-name kv-myapp-dev-XXX \
--name 'DatabasePassword' \
--value 'MySecretP@ss!'

# Read a secret
az keyvault secret show \
--vault-name kv-myapp-dev-XXX \
--name 'DatabasePassword' \
--query value -o tsv

# List all secrets
az keyvault secret list \
--vault-name kv-myapp-dev-XXX \
--output table

# Give an app access to Key Vault (using managed identity)
az keyvault set-policy \
--name kv-myapp-dev-XXX \
--object-id <managed-identity-object-id> \
```

```
--secret-permissions get list
```

17.3 Security Best Practices Checklist

- ■ Enable MFA for all users, especially admins.
- ■ Use Azure Key Vault for ALL secrets — never in code.
- ■ Apply Principle of Least Privilege for RBAC.
- ■ Enable Azure Defender for Cloud and act on recommendations.
- ■ Use Private Endpoints to keep database traffic off the public internet.
- ■ Enable Diagnostic Logs on all resources.
- ■ Set up Azure Policy to enforce company standards.
- ■ Regular security reviews — check your Secure Score monthly.
- ■ Enable Just-In-Time (JIT) VM access instead of always-open ports.
- ■ Use Azure Bastion for SSH/RDP instead of public IPs.

PART 7

Monitoring & Management

Watching your applications and keeping them healthy

Ch 18: Application Insights

Ch 19: Azure Monitor

Ch 20: Azure Dashboards

CHAPTER 18

Application Insights

X-ray vision for your running application! ■■

18.1 What is Application Insights?

Application Insights is an Application Performance Management (APM) service. It automatically monitors your web app's performance, detects failures, and helps you understand what users are actually doing. It's like a doctor for your app!

- **Request tracking** — See every HTTP request to your app: URL, response time, success/fail.
- **Failure detection** — Automatic alerts when exceptions or errors spike.
- **Performance metrics** — CPU, memory, response times, database query times.
- **User behaviour** — Which pages are most visited? Where do users drop off?
- **Live metrics** — Real-time dashboard showing requests and failures as they happen!
- **Distributed tracing** — Follow a request across multiple microservices.

18.2 Setting Up Application Insights — Portal

1	Search Application Insights Search 'Application Insights' in the portal and click '+ Create'.
2	Basics Resource group: rg-myapp-dev. Name: appi-myapp-dev. Region: Southeast Asia. Resource mode: Workspace-based.
3	Log Analytics Workspace Create new workspace: law-myapp-dev. This is where logs are stored.
4	Review + Create Click Create. Takes about 30 seconds.
5	Get your connection string Go to your Application Insights resource. Copy the 'Connection String' from the Overview page.
6	Add to your app Set it as an environment variable or app setting: APPLICATIONINSIGHTS_CONNECTION_STRING=

18.3 Connect App Insights to App Service — CLI

```
AZURE CLI — Application Insights
# Create a Log Analytics workspace
az monitor log-analytics workspace create \
--resource-group rg-myapp-dev \
--workspace-name law-myapp-dev

# Get workspace ID
WORKSPACE_ID=$(az monitor log-analytics workspace show \
--resource-group rg-myapp-dev \
--workspace-name law-myapp-dev \
--query id -o tsv)

# Create Application Insights
az monitor app-insights component create \
--app appi-myapp-dev \
--resource-group rg-myapp-dev \
--location southeastasia \
--workspace $WORKSPACE_ID

# Get the connection string
az monitor app-insights component show \
--app appi-myapp-dev \
--resource-group rg-myapp-dev \
--query connectionString -o tsv

# Set it on your App Service
az webapp config appsettings set \
--resource-group rg-myapp-dev \
--name mywebapp-demo-dev \
--settings APPLICATIONINSIGHTS_CONNECTION_STRING='<your-connection-string>'
```

CHAPTER 19

Azure Monitor

The heartbeat monitor for your entire Azure environment! ■

19.1 Azure Monitor Overview

Azure Monitor is the central platform for collecting, analysing, and acting on telemetry from your Azure resources. Think of it as mission control — all signals from all resources flow through here.

- **Metrics** — Numeric time-series data (CPU %, memory usage, request count).
- **Logs** — Text events stored in Log Analytics (errors, warnings, audit events).
- **Alerts** — Notify you when something crosses a threshold (CPU > 90%).
- **Action Groups** — What to do when an alert fires (email, SMS, Azure Function, ITSM).
- **Workbooks** — Interactive reports and dashboards.

19.2 Creating an Alert — Portal

1	Go to Azure Monitor Search 'Monitor' in the portal.
2	Click Alerts → + Create → Alert rule Click '+ Create' then 'Alert rule'.
3	Select the resource Click 'Select scope'. Find your VM or App Service. Click 'Done'.
4	Set the condition Click 'Add condition'. Search 'Percentage CPU'. Set threshold: Operator: Greater than. Threshold: 80. Aggregation: Average. Period: 5 minutes.
5	Create action group Click 'Create action group'. Name: ag-email-dev. Add action: Email. Enter your email. Click 'OK'.
6	Alert details Alert rule name: alert-high-cpu. Severity: 2 (Warning). Click 'Review + create'.

19.3 Log Analytics Queries (KQL)

Kusto Query Language (KQL) is the query language for Log Analytics. It's powerful and readable!

KQL – Log Analytics Queries

```
// View all requests in App Insights (last 1 hour)
```

```
requests
```

```
| where timestamp > ago(1h)
```

```
| summarize count() by resultCode
```

```
| render barchart
```

```
// Find all exceptions
```

```
exceptions
```

```
| where timestamp > ago(24h)
```

```
| project timestamp, type, outerMessage
```

```
| order by timestamp desc
```

```
// Average response time by URL
```

```
requests
```

```
| where timestamp > ago(1h)
```

```
| summarize avg(duration) by name
```

```
| order by avg_duration desc
```

```
// VM CPU over time
```

```
Perf
```

```
| where ObjectName == 'Processor' and CounterName == '% Processor Time'
```

```
| where TimeGenerated > ago(1h)
```

```
| summarize avg(CounterValue) by Computer, bin(TimeGenerated, 5m)
```

```
| render timechart
```

CHAPTER 20

Azure Dashboards

Build your perfect mission control view! ■■

20.1 What are Azure Dashboards?

Azure Dashboards are customisable views in the Azure Portal where you pin charts, metrics, logs, resource health, and other tiles. Perfect for creating an at-a-glance view of your entire application.

20.2 Creating a Dashboard — Portal

1	Go to Dashboard Click 'Dashboard' in the portal's left menu. Click '+ New dashboard' → 'Blank dashboard'.
2	Name it Type a name like 'MyApp Production Dashboard'.
3	Add tiles From the Tile Gallery on the right, drag tiles onto the dashboard. Try: Metrics chart, Markdown (for notes), Clock, Resource health.
4	Pin from resources Navigate to any resource (VM, App Service). Any chart you see has a pin icon (■). Click it to pin to your dashboard!
5	Share the dashboard Click 'Share' at the top. Publish to a resource group so your team can see it.
6	Save Click 'Done customising' to save.

20.3 Recommended Dashboard Tiles

- **Metrics chart** — CPU, memory, requests for your VMs and App Services.
- **Application Insights — Server response time** — Is your app slow?
- **Application Insights — Failed requests** — Are there errors?
- **Resource health** — Is Azure having issues with your resources?
- **Azure Advisor Score** — Recommendations for cost, performance, security.
- **Cost Management — Current spend** — How much are you spending?

■ **TIP** Export your dashboard as JSON and store it in Git! Then you can recreate it anytime with one CLI command. This is 'Infrastructure as Code' for dashboards.

PART 8

DevOps & Pipelines

Automating everything from code to cloud

Ch 21: Azure DevOps Overview

Ch 22: Azure Repos

Ch 23: Azure Boards

Ch 24: Azure YAML Pipelines

CHAPTER 21

Azure DevOps Overview

The complete factory for building and shipping software! ■

■ ■ **ANALOGY:** Azure DevOps is like a car factory. Raw materials (code) go in one end. After testing, quality checks, and assembly (CI/CD pipeline), a finished car (deployed app) comes out the other end — automatically!

21.1 Azure DevOps Services

Service	What It Does	Like...
Azure Repos	Git source code hosting	GitHub
Azure Boards	Work item tracking (stories, tasks, bugs)	Jira / Trello
Azure Pipelines	CI/CD automation (build, test, deploy)	Jenkins / GitHub Actions
Azure Artifacts	Package management (npm, NuGet, pip)	npm Registry
Azure Test Plans	Manual and automated test management	TestRail

21.2 Getting Started with Azure DevOps

1	Create account Go to dev.azure.com. Sign in with your Microsoft account. Click 'New organization'.
2	Create a project Click '+ New project'. Name: 'MyAwesomeApp'. Visibility: Private. Version control: Git. Work item process: Agile. Click 'Create project'.
3	Explore the menu Left sidebar: Overview, Boards, Repos, Pipelines, Test Plans, Artifacts.

21.3 DevOps vs GitHub

Feature	Azure DevOps	GitHub
Code hosting	Azure Repos	GitHub Repos
CI/CD	Azure Pipelines	GitHub Actions
Work tracking	Azure Boards	GitHub Issues / Projects
Packages	Azure Artifacts	GitHub Packages
Best for	Enterprise teams, full ALM	Open source, developer community

■ **TIP** Good news — Azure Pipelines works with both Azure Repos AND GitHub! You can keep your code in GitHub and use Azure Pipelines to deploy to Azure.

CHAPTER 22

Azure Repos

Your code's home — version control in the cloud! ■

22.1 Git Basics Refresher

- **Repository (repo)** — A folder containing your project and its history.
- **Commit** — A snapshot of your changes, like saving a version.
- **Branch** — A parallel version of the code (main, feature/login, bugfix/123).
- **Pull Request (PR)** — A request to merge your branch into main, with code review.
- **Clone** — Download a copy of the repo to your computer.

22.2 Working with Azure Repos — CLI

```
GIT + AZURE REPOS
# Clone your Azure Repo
git clone https://dev.azure.com/yourorg/MyAwesomeApp/_git/MyAwesomeApp
cd MyAwesomeApp

# Create a new feature branch
git checkout -b feature/add-login-page

# Make changes... then stage and commit
git add .
git commit -m 'feat: add login page with Azure AD integration'

# Push to Azure Repos
git push origin feature/add-login-page

# Create a Pull Request (use the Azure DevOps web UI or:
az repos pr create \
  --repository MyAwesomeApp \
  --source-branch feature/add-login-page \
  --target-branch main \
  --title 'Add login page' \
  --description 'Implements login with Azure AD' \
  --auto-complete true
```

22.3 Branch Policies

Branch policies protect your main branch from direct pushes and ensure code quality:

- Require a minimum number of reviewers (e.g., 2 approvals before merge).
- Require linked work items (every PR must reference an Azure Boards task).
- Require build to pass (pipeline must succeed before merging).

- Block force pushes (nobody can overwrite history).

1	Go to Repos In Azure DevOps, click 'Repos' → 'Branches'.
2	Set policy on main Click the '...' menu next to 'main' → 'Branch policies'.
3	Enable policies Turn on: Require a minimum number of reviewers (set to 1 or 2). Require a successful build. Click Save.

CHAPTER 23

Azure Boards

Your project's to-do list on steroids! ■

■ ■ **ANALOGY:** Azure Boards is like a surfing competition planner. You have a big wall (board) with sticky notes (work items) that move from 'To Do' → 'In Progress' → 'Done' as your team does work.

23.1 Work Item Types (Agile Process)

Type	Description	Example
Epic	Large body of work spanning months	User Authentication System
Feature	A chunk of an Epic, weeks of work	Login Page, Password Reset
User Story	A user-facing feature, days of work	As a user, I can log in with email
Task	Technical implementation steps, hours	Create login form HTML
Bug	Something broken that needs fixing	Login fails with special chars

23.2 Sprints & Backlog

- **Product Backlog** — All the work that needs to be done (prioritised list).
- **Sprint Backlog** — Work chosen for this sprint (usually 2 weeks).
- **Sprint Board** — Visual Kanban board: To Do → In Progress → Done.
- **Sprint Review** — End of sprint: demo what was built.
- **Retrospective** — What went well? What to improve?

23.3 Creating Work Items — Portal

1

Go to Boards → Backlogs

In your project, click 'Boards' → 'Backlogs'.

2

Create an Epic

Click '+ New Work Item' → select 'Epic'. Title: 'User Authentication'. Press Enter.

3	Create a User Story Click the '+' under the Epic. Select 'Feature'. Add: 'Login Page'. Then add User Story: 'As a user, I want to log in with my email so I can access my account'.
4	Add Tasks Click the User Story → click '+ Add link' → 'Child'. Add tasks: 'Design login form', 'Implement API endpoint', 'Write unit tests'.
5	Move to Sprint Select your User Story. Click '...' → 'Move to Sprint' → 'Sprint 1'.
6	View the board Click 'Boards' → 'Boards'. You'll see your sprint board! Drag cards to change status.

CHAPTER 24

Azure YAML Pipelines

Teaching your pipeline to build, test, and deploy automatically! ■

■ ■ **ANALOGY:** A YAML pipeline is like a robot chef recipe. You write the steps once: 'chop vegetables → cook → plate → serve'. The robot follows it exactly every time, never forgets a step, and works 24/7!

24.1 YAML Pipeline Structure

```
AZURE YAML PIPELINE – Full Example
# azure-pipelines.yml – basic structure
trigger: # When to run
branches:
include:
- main
- feature/*

pool: # Where to run
vmImage: ubuntu-latest # Microsoft-hosted agent

variables: # Reusable values
buildConfiguration: Release
appName: mywebapp-demo-dev

stages: # Top-level phases
- stage: Build
  displayName: 'Build Stage'
  jobs:
  - job: BuildApp
    displayName: 'Build Application'
    steps:
    - task: NodeTool@0
      inputs:
        versionSpec: '18.x'
      displayName: 'Install Node.js'
    - script: |
        npm install
        npm run build
        npm test
      displayName: 'Install, Build & Test'
    - task: PublishBuildArtifacts@1
      inputs:
        pathToPublish: '$(Build.ArtifactStagingDirectory)'
        artifactName: 'drop'
```

```

displayName: 'Publish Artifacts'
- stage: Deploy
  displayName: 'Deploy to Dev'
  dependsOn: Build
  condition: succeeded()
  jobs:
  - deployment: DeployWebApp
    displayName: 'Deploy to App Service'
    environment: 'development'
    strategy:
      runOnce:
        deploy:
          steps:
          - task: AzureWebApp@1
            inputs:
              azureSubscription: 'sc-azure-myapp'
              appType: webAppLinux
              appName: $(appName)
              package: '$(Pipeline.Workspace)/drop/**/*.*.zip'

```

24.2 Common Pipeline Tasks

Task	Purpose	YAML Key
NodeTool@0	Install Node.js	task: NodeTool@0
DotNetCoreCLI@2	Build/test .NET	task: DotNetCoreCLI@2
Docker@2	Build/push Docker images	task: Docker@2
AzureWebApp@1	Deploy to App Service	task: AzureWebApp@1
AzureCLI@2	Run Azure CLI commands	task: AzureCLI@2
CopyFiles@2	Copy files between dirs	task: CopyFiles@2
PublishTestResults@2	Publish test results	task: PublishTestResults@2

24.3 Setting Up Your First Pipeline

1 Go to Pipelines

In Azure DevOps, click 'Pipelines' → '+ New pipeline'.

2 Connect to code

Choose 'Azure Repos Git'. Select your repository.

3	Configure Choose 'Starter pipeline' (we'll edit it) or 'Existing YAML file' if you already have one.
4	Edit the YAML Replace the generated YAML with your pipeline code. Click 'Save and run'.
5	Watch it run! See each stage and step execute in real time. Green checkmarks = success! Red X = failure — click to see the error log.
6	Create service connection If deploying to Azure, you'll need to create a service connection (see Chapter 16). The wizard will prompt you.

PART 9

Capstone Project

Build a real 3-Tier Architecture — Portal & CLI

Ch 25: 3-Tier Architecture — Portal Walkthrough

Ch 26: 3-Tier Architecture — Azure CLI Script

CHAPTER 25

Capstone: 3-Tier Architecture — Portal

Build a real production-ready application from scratch! ■■

In this capstone, we build a classic 3-Tier web application: a frontend (web tier), a backend API (app tier), and a database (data tier). This is the architecture used by real companies worldwide!

Architecture Overview

Tier	Azure Service	Subnet	Purpose
Web/Frontend	Azure App Service	subnet-web (10.0.1.0/24)	Serve the user interface
App/Backend	Azure App Service	subnet-app (10.0.2.0/24)	Business logic & APIs
Data/Database	Azure SQL Database	subnet-db (10.0.3.0/24)	Persistent data storage

Step 1 — Create the Resource Group

- 1 Portal**
Search 'Resource groups' → '+ Create'. Name: rg-threetier-prod. Region: East US. Tags: Environment=Production, Project=ThreeTier. Click 'Review + create' → 'Create'.

Step 2 — Create the Virtual Network

- 1 Create VNet**
Search 'Virtual networks' → '+ Create'. Name: vnet-threetier-prod. Address space: 10.0.0.0/16. Resource group: rg-threetier-prod.
- 2 Add subnets**
In IP Addresses tab, add 3 subnets: subnet-web (10.0.1.0/24), subnet-app (10.0.2.0/24), subnet-db (10.0.3.0/24). Click 'Review + create'.

Step 3 — Create NSGs

- 1 NSG for Web tier**
Search 'Network security groups' → '+ Create'. Name: nsg-web. Resource group: rg-threetier-prod.

2	Add inbound rules to nsg-web Rule 1: Allow-HTTP — Priority 100, Source: Any, Port: 80, Allow. Rule 2: Allow-HTTPS — Priority 110, Port: 443, Allow.
3	NSG for App tier Create nsg-app. Add rule: Allow-from-Web — Priority 100, Source: 10.0.1.0/24, Port: 8080, Allow.
4	NSG for DB tier Create nsg-db. Add rule: Allow-from-App — Priority 100, Source: 10.0.2.0/24, Port: 1433, Allow. Add Deny-All-Inbound: Priority 4000, Source: Any, Port: Any, Deny.
5	Attach NSGs to subnets Go to each subnet in the VNet, click it, and associate the appropriate NSG.

Step 4 — Create the Database Tier

1	Create SQL Server Search 'SQL servers' → '+ Create'. Name: sql-threetier-prod-XXX. Resource group: rg-threetier-prod. Admin: sqladmin / StrongP@ss123!
2	Create Database In the SQL server, click '+ Create database'. Name: db-threetier-prod. Compute: Basic (5 DTUs for testing).
3	Configure networking In the SQL Server → Networking, add a Private Endpoint in subnet-db. This keeps all DB traffic private!

Step 5 — Create the Backend App Service (API Tier)

1	Create App Service Plan Search 'App Service plans' → '+ Create'. Name: asp-api-prod. OS: Linux. SKU: Standard S1 (needed for VNet integration).
2	Create Web App Search 'App Services' → '+ Create'. Name: api-threetier-prod. Runtime: Node 18. Plan: asp-api-prod.
3	Enable VNet Integration In the App Service → Networking → VNet Integration → 'Add VNet integration'. VNet: vnet-threetier-prod. Subnet: subnet-app.

4 Set connection string

In App Service → Configuration → Connection strings. Add: Name: DefaultConnection. Value: your SQL connection string. Click Save.

Step 6 — Create the Frontend App Service (Web Tier)

1 Create App Service Plan

Create asp-web-prod. OS: Linux. SKU: Standard S1.

2 Create Web App

Name: web-threetier-prod. Runtime: Node 18. Plan: asp-web-prod.

3 Enable VNet Integration

VNet: vnet-threetier-prod. Subnet: subnet-web.

4 Set API URL

Configuration → App settings. Add: API_URL = https://api-threetier-prod.azurewebsites.net

Step 7 — Add Application Insights

1 Create App Insights

Create appi-threetier-prod in rg-threetier-prod.

2 Connect to both App Services

On each App Service → Application Insights → Turn on Application Insights → select appi-threetier-prod.

Step 8 — Test Your Architecture

1 Get the frontend URL

Go to web-threetier-prod App Service. Copy the URL from Overview.

2 Test in browser

Open the URL. The frontend should load. Any API calls go through the VNet to the backend!

3 Check Application Insights

Open appi-threetier-prod → Live Metrics. You'll see real-time traffic!

4 Check Metrics

Go to each App Service → Metrics. Pin CPU and Request count to your dashboard.

Capstone: 3-Tier Architecture — Azure CLI

■ **TIP** Copy the script below into Azure Cloud Shell and run it section by section. The entire 3-tier architecture will be deployed in about 15 minutes!

```
COMPLETE 3-TIER DEPLOYMENT SCRIPT
#!/bin/bash
# ████████████████████████████████████████████████████████████████
# 3-TIER AZURE ARCHITECTURE – COMPLETE CLI SCRIPT
# ████████████████████████████████████████████████████████████████
# ■■ VARIABLES ████████████████████████████████████████████████████████████████
RG='rg-threetier-prod'
LOCATION='eastus'
VNET='vnet-threetier-prod'
SQL_SERVER='sql-threetier-prod-001'
SQL_ADMIN='sqladmin'
SQL_PASS='StrongP@ss123!'
DB_NAME='db-threetier-prod'
API_APP='api-threetier-prod'
WEB_APP='web-threetier-prod'
ASP_API='asp-api-prod'
ASP_WEB='asp-web-prod'
APP_INSIGHTS='appi-threetier-prod'
LOG_WORKSPACE='law-threetier-prod'

echo '■ Starting 3-Tier Architecture Deployment...'

# ■■ STEP 1: Resource Group ████████████████████████████████████████████████████████████████
echo '■ Creating Resource Group...'

az group create \
--name $RG \
--location $LOCATION \
--tags Environment=Production Project=ThreeTier

# ■■ STEP 2: Virtual Network ████████████████████████████████████████████████████████████████
echo '■ Creating VNet...'

az network vnet create \
--resource-group $RG \
--name $VNET \
--address-prefix 10.0.0.0/16

# Create subnets
for SUBNET_NAME in 'subnet-web' 'subnet-app' 'subnet-db'; do
case $SUBNET_NAME in
subnet-web) PREFIX='10.0.1.0/24' ;;
```

```
subnet-app) PREFIX='10.0.2.0/24' ;;  
subnet-db) PREFIX='10.0.3.0/24' ;;  
  
esac  
  
az network vnet subnet create \  
--resource-group $RG \  
--vnet-name $VNET \  
--name $SUBNET_NAME \  
--address-prefix $PREFIX  
  
echo " ■ Created $SUBNET_NAME ($PREFIX)"  
  
done  
  
# ■■ STEP 3: Network Security Groups ██████████  
  
echo '■ Creating NSGs...'  
  
# Web NSG  
  
az network nsg create --resource-group $RG --name nsg-web  
az network nsg rule create --resource-group $RG --nsg-name nsg-web \  
--name Allow-HTTP --priority 100 --direction Inbound \  
--source-address-prefixes Internet --destination-port-ranges 80 \  
--protocol Tcp --access Allow  
  
az network nsg rule create --resource-group $RG --nsg-name nsg-web \  
--name Allow-HTTPS --priority 110 --direction Inbound \  
--source-address-prefixes Internet --destination-port-ranges 443 \  
--protocol Tcp --access Allow  
  
# App NSG  
  
az network nsg create --resource-group $RG --name nsg-app  
az network nsg rule create --resource-group $RG --nsg-name nsg-app \  
--name Allow-From-Web --priority 100 --direction Inbound \  
--source-address-prefixes 10.0.1.0/24 --destination-port-ranges 8080 \  
--protocol Tcp --access Allow  
  
# DB NSG  
  
az network nsg create --resource-group $RG --name nsg-db  
az network nsg rule create --resource-group $RG --nsg-name nsg-db \  
--name Allow-From-App --priority 100 --direction Inbound \  
--source-address-prefixes 10.0.2.0/24 --destination-port-ranges 1433 \  
--protocol Tcp --access Allow  
  
# Attach NSGs to subnets  
  
az network vnet subnet update --resource-group $RG --vnet-name $VNET \  
--name subnet-web --network-security-group nsg-web  
az network vnet subnet update --resource-group $RG --vnet-name $VNET \  
--name subnet-app --network-security-group nsg-app  
az network vnet subnet update --resource-group $RG --vnet-name $VNET \  
--name subnet-db --network-security-group nsg-db  
  
# ■■ STEP 4: SQL Database ██████████  
  
echo '■■ Creating SQL Server & Database...'  
  
az sql server create \  
--name $SQL_SERVER \  
--resource-group $RG \  
--location $LOCATION \
```



```
--admin-user $SQL_ADMIN \
--admin-password $SQL_PASS

az sql db create \
--resource-group $RG \
--server $SQL_SERVER \
--name $DB_NAME \
--edition Basic \
--capacity 5

# Allow Azure services to connect
az sql server firewall-rule create \
--resource-group $RG \
--server $SQL_SERVER \
--name AllowAzureServices \
--start-ip-address 0.0.0.0 \
--end-ip-address 0.0.0.0

# Get the connection string
SQL_CONN="Server=tcp:${SQL_SERVER}.database.windows.net,1433;Database=${DB_NAME};
User ID=${SQL_ADMIN};Password=${SQL_PASS};Encrypt=true;"

# ■■■ STEP 5: App Service Plans ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
echo '■■ Creating App Service Plans...'

az appservice plan create \
--name $ASP_API \
--resource-group $RG \
--sku S1 --is-linux

az appservice plan create \
--name $ASP_WEB \
--resource-group $RG \
--sku S1 --is-linux

# ■■■ STEP 6: Web Apps ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
echo '■ Creating Web Apps...'

az webapp create \
--resource-group $RG \
--plan $ASP_API \
--name $API_APP \
--runtime 'NODE:18-lts'

az webapp create \
--resource-group $RG \
--plan $ASP_WEB \
--name $WEB_APP \
--runtime 'NODE:18-lts'

# Set database connection string on API app
az webapp config connection-string set \
--resource-group $RG \
--name $API_APP \
--connection-string-type SQLAzure \
--settings DefaultConnection="$SQL_CONN"

# Set API URL on Web app
```

[illegible]

Testing Your 3-Tier Architecture

1 Open frontend

Go to <https://web-threetier-prod.azurewebsites.net> in your browser.

2	Test API directly Open <code>https://api-threetier-prod.azurewebsites.net/health</code> — you should get a 200 OK response.
3	Check Application Insights Open <code>appi-threetier-prod</code> → Live Metrics. Make some requests and watch them appear in real time!
4	Run KQL query In App Insights → Logs, run: <code>requests summarize count() by resultCode render piechart</code>
5	Check costs Go to Cost Management → Cost analysis. Filter by resource group <code>rg-threetier-prod</code> . See your spending!
6	Clean up Run: <code>az group delete --name rg-threetier-prod --yes --no-wait</code>

■■ **IMPORTANT** IMPORTANT: When you finish testing, always delete the resource group to stop all charges! The command above deletes everything in one go.

CHAPTER QR

Quick Reference Card

All the most important commands in one place!

Azure CLI Cheat Sheet

AZURE CLI QUICK REFERENCE

AUTH

```
az login | az account show | az account list
```

RESOURCE GROUPS

```
az group create --name NAME --location REGION
```

```
az group list --output table
```

```
az group delete --name NAME --yes
```

VMs

```
az vm create -g RG -n NAME --image Ubuntu2204 --generate-ssh-keys
```

```
az vm stop/start/deallocate/delete -g RG -n NAME
```

APP SERVICE

```
az appservice plan create -g RG -n NAME --sku FREE --is-linux
```

```
az webapp create -g RG --plan PLAN -n NAME --runtime 'NODE:18-lts'
```

```
az webapp up -g RG -n NAME --html
```

STORAGE

```
az storage account create -n NAME -g RG --sku Standard_LRS
```

```
az storage container create --account-name ACCT -n CONTAINER
```

```
az storage blob upload --account-name ACCT -c CONTAINER -n FILE -f /path
```

SQL

```
az sql server create -n NAME -g RG -l REGION -u ADMIN -p PASS
```

```
az sql db create -g RG -s SERVER -n DB --edition Basic
```

NETWORKING

```
az network vnet create -g RG -n NAME --address-prefix 10.0.0.0/16
```

```
az network vnet subnet create -g RG --vnet-name VNET -n NAME --address-prefix 10.0.1.0/24
```

```
az network nsg create -g RG -n NAME
```

KEY VAULT

```
az keyvault create -n NAME -g RG -l REGION
```

```
az keyvault secret set --vault-name VAULT -n SECRET_NAME --value VALUE
```

```
az keyvault secret show --vault-name VAULT -n SECRET_NAME --query value
```

MONITORING

```
az monitor app-insights component create --app NAME -g RG -l REGION
```

IDENTITY

```
az ad user create --display-name NAME --user-principal-name UPN --password PASS
```

```
az ad sp create-for-rbac --name NAME --role Contributor --scopes /subscriptions/SUB_ID
```

```
az role assignment create --assignee EMAIL --role ROLE -g RG
```

Naming Conventions

Resource	Convention	Example
Resource Group	rg-{app}-{env}	rg-webshop-prod
Virtual Network	vnet-{app}-{env}	vnet-webshop-prod
Subnet	subnet-{tier}	subnet-web
NSG	nsg-{tier}	nsg-web
VM	vm-{role}-{num}	vm-webserver-01
App Service	{app}-{env}	webshop-prod
SQL Server	sql-{app}-{env}-XXX	sql-webshop-prod-001
Storage	{app}{env}XXX	webshoapprod001
Key Vault	kv-{app}-{env}-XXX	kv-webshop-prod-001
App Insights	appi-{app}-{env}	appi-webshop-prod